

ETEC FERNANDO PRESTES
3º AMS DESENVOLVIMENTO DE SISTEMAS

DOCUMENTAÇÃO TÉCNICA DO SISTEMA DE E-COMMERCE
ARQUITETURA MERN FULL STACK WEB
PROGRAMAÇÃO DE APLICATIVOS MOBILE - PAM

Alan Marcelo Bravo
Benício Rodrigues Ferreira
Henrique Furquim
Thalles Yuji Gambary

SOROCABA - SP
2026

ALAN MARCELO BRAVO
BENÍCIO RODRIGUES FERREIRA
HENRIQUE FURQUIM
THALLES YUJI GAMBARY

**DOCUMENTAÇÃO TÉCNICA DO SISTEMA DE E-COMMERCE
ARQUITETURA MERN FULL STACK WEB**

Documentação técnica apresentada à disciplina de Programação de Aplicativos Mobile (PAM), do curso 3º AMS Desenvolvimento de Sistemas da Etec Fernando Prestes, como parte da atividade de Arquitetura e Desenvolvimento MERN Full Stack Web, sob orientação do professor Benir Falcão Pistille.

SOROCABA - SP
2026

SUMÁRIO

1 INTRODUÇÃO	6
2 OBJETIVOS	6
3 ANÁLISE DA ATIVIDADE SOLICITADA	6
4 LEVANTAMENTO TÉCNICO DO SISTEMA	7
5 TECNOLOGIAS UTILIZADAS	8
6 ARQUITETURA DO SISTEMA	8
7 BANCO DE DADOS	9
8 ROTAS E ENDPOINTS DA API	10
9 INTERFACE E TELAS DO SISTEMA	10
10 TESTES E VALIDAÇÃO	11
11 FLUXOGRAMA LÓGICO DE DEVOPS	11
12 COMO EXECUTAR O PROJETO	13
13 ANÁLISE FINAL: O QUE FALTA OU PODE MELHORAR	13
14 CONCLUSÃO	14
REFERÊNCIAS	14
APÊNDICE A - CÓDIGO-FONTE PRINCIPAL DA SOLUÇÃO	15

1 INTRODUÇÃO

Este documento apresenta a documentação técnica do sistema de e-commerce desenvolvido pela equipe para a atividade de Arquitetura e Desenvolvimento MERN Full Stack Web. O trabalho foi analisado a partir do arquivo do projeto enviado em formato ZIP e do PDF com as orientações da atividade. A proposta solicitada consiste no desenvolvimento de um software de e-commerce, acompanhado de documentação do código e de um fluxograma lógico de DevOps para representar o processo de automação e entrega da solução.

O sistema analisado possui arquitetura baseada em frontend React com Vite, backend Node.js com Express, persistência de dados em MongoDB Atlas e comunicação entre camadas por meio de API REST. A aplicação permite cadastrar usuários, realizar login, listar produtos, visualizar detalhes de produtos, gerenciar produtos e usuários em um painel administrativo e atualizar dados de perfil.

2 OBJETIVOS

2.1 Objetivo geral

Documentar, em formato técnico e organizado, o sistema de e-commerce desenvolvido pela equipe, descrevendo sua arquitetura, tecnologias, funcionalidades, estrutura de código, banco de dados, API, testes e fluxo lógico de DevOps.

2.2 Objetivos específicos

- Apresentar o levantamento técnico do sistema desenvolvido.
- Descrever as camadas frontend, backend e banco de dados da aplicação.
- Registrar as principais funcionalidades implementadas no e-commerce.
- Anexar a codificação principal da solução, excluindo dependências de terceiros e dados sensíveis.
- Propor um fluxo lógico de DevOps com etapas de versionamento, testes, build, deploy, monitoramento e feedback.
- Verificar a aderência do projeto aos itens solicitados na atividade.

3 ANÁLISE DA ATIVIDADE SOLICITADA

O PDF da atividade orienta a equipe a desenvolver um software de e-commerce com fluxograma lógico de DevOps, considerando o papel da fábrica de software na criação de um ambiente de processos automáticos. A entrega solicitada envolve três partes principais:

aplicativo ou software funcional, documentação técnica com a codificação da solução e desenho do fluxograma lógico de DevOps com teste e monitoração em tempo real.

Com base nesses requisitos, a documentação foi estruturada para cobrir a solução implementada no ZIP, explicar a arquitetura utilizada e complementar o trabalho com um fluxo de DevOps coerente com o projeto. Como o código enviado não contém arquivos de pipeline CI/CD nem scripts de testes automatizados, esses pontos são tratados como proposta de melhoria e como itens pendentes caso o professor exija implementação real dessas etapas.

Item solicitado	Situação no projeto	Observação
Software de e-commerce	Implementado parcialmente	Há frontend, backend, cadastro/login, produtos, detalhe e painel ADM.
Documentação do código	Atendido neste documento	A codificação principal foi anexada em apêndice, sem <code>node_modules</code> e sem <code>.env</code> real.
Fluxograma lógico de DevOps	Atendido como documentação/proposta	O fluxo foi desenhado e descrito; não há pipeline CI/CD real no ZIP.
Teste e monitoramento	Parcial	Há proposta de testes e monitoramento, mas não há ferramenta automatizada configurada no projeto.

4 LEVANTAMENTO TÉCNICO DO SISTEMA

O sistema é um e-commerce chamado PAM - E-commerce. Ele foi construído para permitir que usuários acessem uma loja virtual, realizem cadastro, façam login, visualizem produtos, consultem detalhes de cada produto e acessem um painel administrativo para gerenciamento de produtos e usuários. A aplicação utiliza uma separação clara entre frontend e backend.

4.1 Usuários e perfis

O modelo de usuário possui os campos nome, nome de usuário, e-mail, senha, endereço, telefone, CEP e permissão. A permissão pode ser comum ou admin. A senha é armazenada no banco após aplicação de hash com `bcryptjs`, o que aumenta a segurança em relação ao armazenamento de texto puro.

4.2 Produtos

O modelo de produto contém nome, descrição, preço, imagem e estoque. A página principal da loja consome a API de produtos e exibe os itens cadastrados em formato de vitrine. A página de detalhe apresenta descrição, imagem, preço, estoque e opção de compra simulada.

4.3 Funcionalidades identificadas

- Página inicial com identificação da disciplina e integrantes do grupo.
- Página Home com listagem dos produtos vindos da API.
- Cadastro de usuário com validação de senha e confirmação.
- Login com autenticação por e-mail e senha.
- Armazenamento do usuário logado em sessionStorage.
- Perfil do usuário com consulta e atualização de dados.
- Detalhe de produto com estoque e compra simulada.
- Painel administrativo para criar, listar, editar e deletar produtos.
- Painel administrativo para listar e excluir usuários.

5 TECNOLOGIAS UTILIZADAS

Camada	Tecnologias	Função
Frontend	React, Vite, React Router DOM, Lucide React	Interface, rotas e componentes visuais.
Backend	Node.js, Express, CORS, dotenv	Servidor HTTP e API REST.
Banco de dados	MongoDB Atlas, Mongoose	Persistência em nuvem e modelagem dos dados.
Segurança	bcryptjs	Hash de senha no cadastro e validação no login.
Comunicação	fetch API, JSON	Integração entre frontend e backend.
DevOps proposto	Git/GitHub, CI, build, deploy, logs e monitoramento	Fluxo lógico de automação e melhoria contínua.

6 ARQUITETURA DO SISTEMA

A aplicação segue uma arquitetura em camadas. O usuário interage com o frontend React, que executa rotas e páginas no navegador. O frontend consome a API REST hospedada no backend Node.js/Express. O backend concentra as regras de acesso aos dados, valida entradas, aplica hash de senha e se comunica com o MongoDB Atlas por meio do Mongoose.

Arquitetura do Sistema MERN - E-commerce PAM

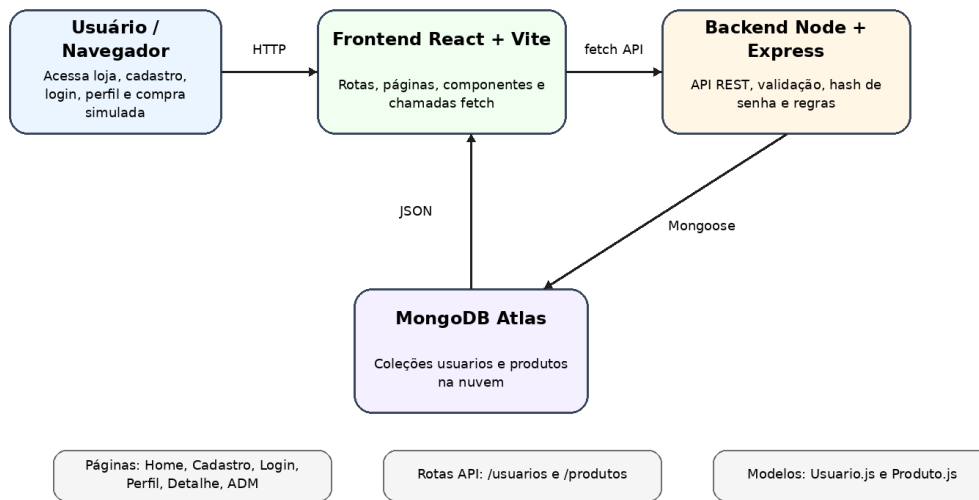


Figura 1 - Diagrama de arquitetura do sistema MERN.

6.1 Fluxo de comunicação

- O usuário acessa o frontend pelo navegador.
- O React Router define qual página será exibida.
- As páginas chamam o arquivo services/api.js para consumir endpoints REST.
- O backend Express recebe a requisição, valida os dados e executa operações nos modelos Mongoose.
- O MongoDB Atlas armazena usuários e produtos.
- A resposta retorna ao frontend em JSON para atualização da interface.

7 BANCO DE DADOS

O banco de dados utilizado é o MongoDB Atlas. A conexão é realizada no arquivo server.js por meio da variável de ambiente MONGO_URI. O projeto contém dois modelos principais: Usuario e Produto.

7.1 Modelo Usuario

Campo	Tipo	Regra
nome	String	Obrigatório, com trim
usuario	String	Obrigatório, único, lowercase
email	String	Obrigatório, único, lowercase
senha	String	Obrigatório, mínimo de 6 caracteres e armazenado com hash
endereço	String	Opcional
telefone	String	Opcional
cep	String	Opcional

role	String	comum ou admin, padrão comum
------	--------	------------------------------

7.2 Modelo Produto

Campo	Tipo	Regra
nome	String	Obrigatório
descricao	String	Valor padrão quando vazio
preco	Number	Obrigatório, mínimo 0
imagem	String	URL da imagem, com padrão placeholder
estoque	Number	Padrão 0, mínimo 0

8 ROTAS E ENDPOINTS DA API

Método	Endpoint	Descrição
GET	/api/produtos	Lista todos os produtos cadastrados.
GET	/api/produtos/:id	Busca o detalhe de um produto pelo ID.
POST	/api/produtos	Cria um novo produto.
PUT	/api/produtos/:id	Atualiza um produto existente.
DELETE	/api/produtos/:id	Remove um produto.
POST	/api/usuarios/cadastro	Cadastra um novo usuário.
POST	/api/usuarios/login	Autentica um usuário.
GET	/api/usuarios	Lista todos os usuários sem exibir senha.
GET	/api/usuarios/:id	Busca usuário por ID sem exibir senha.
PUT	/api/usuarios/:id	Atualiza dados de perfil.
DELETE	/api/usuarios/:id	Remove usuário.

9 INTERFACE E TELAS DO SISTEMA

A interface foi construída com componentes React e rotas gerenciadas pelo React Router. A estrutura principal está em App.jsx, que organiza as rotas de página inicial, home da loja, login, cadastro, perfil, detalhe do produto e painel administrativo.

Tela	Função
Página Inicial	Apresenta a matéria PAM, atividade do bimestre e integrantes do grupo.
Home	Exibe os produtos cadastrados no banco em formato de vitrine.
Cadastro	Permite criar uma conta com nome, usuário, endereço, CEP, telefone, e-mail e senha.
Login	Permite autenticação por e-mail e senha e salva o usuário na sessão.
Perfil	Permite visualizar e atualizar dados do usuário logado.
Detalhe do Produto	Apresenta informações de um produto e botão de compra simulada.
Painel ADM	Permite gerenciar produtos e usuários.

10 TESTES E VALIDAÇÃO

A atividade solicita que o processo de DevOps envolva teste e monitoração. O ZIP enviado não possui arquivos de testes automatizados configurados, como Jest, Cypress, Selenium, Robot Framework ou scripts de integração. Portanto, foram definidos testes manuais e recomendações de automação para complementar a entrega.

10.1 Testes manuais sugeridos

Cenário	Procedimento	Resultado esperado
Cadastro válido	Preencher todos os campos e confirmar senha igual	Usuário cadastrado e redirecionamento para login.
Cadastro com senhas diferentes	Informar senha e confirmação diferentes	Sistema deve impedir cadastro.
Login válido	Entrar com usuário cadastrado	Sistema deve salvar sessão e abrir home.
Login inválido	Entrar com senha incorreta	Sistema deve exibir erro.
Listagem de produtos	Abrir home com backend rodando	Produtos do MongoDB devem aparecer.
CRUD de produto	Criar, editar e deletar produto no painel ADM	Produto deve refletir no banco e na loja.
Perfil	Atualizar dados do usuário logado	Dados devem ser salvos pela API.

10.2 Testes automatizados recomendados

- Criar testes de API para validar cadastro, login e CRUD de produtos.
- Criar testes de interface para verificar navegação, formulários e painel administrativo.
- Adicionar script npm test no backend e no frontend.
- Incluir testes no pipeline de DevOps antes do deploy.
- Utilizar logs e monitoramento de disponibilidade para acompanhar falhas em tempo real.

11 FLUXOGRAMA LÓGICO DE DEVOPS

O fluxo de DevOps proposto representa como a fábrica de software poderia automatizar o ciclo de desenvolvimento, teste, entrega e monitoramento do e-commerce. O objetivo é garantir clareza, padronização, eficiência e automação no processo, conforme orientado na atividade.

Fluxograma Lógico de DEVOPS - E-commerce PAM

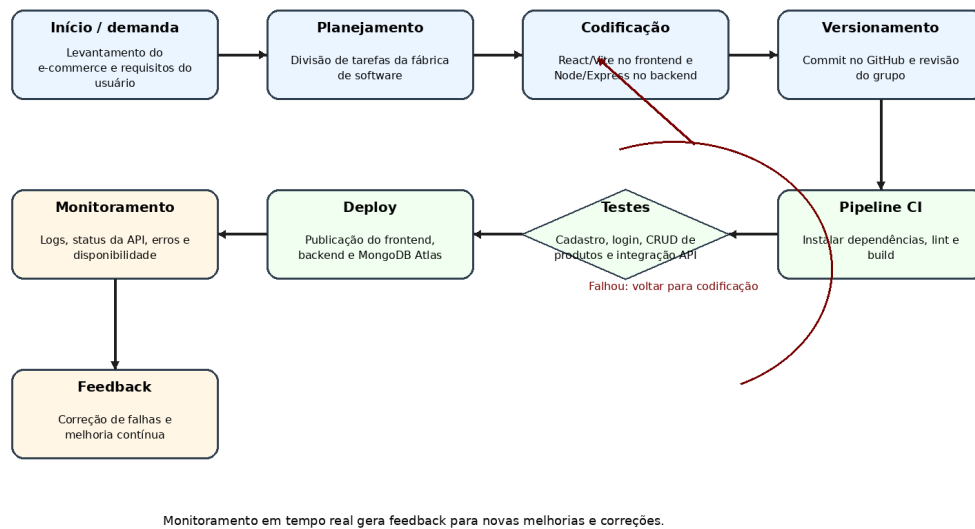


Figura 2 - Fluxograma lógico de DevOps para o e-commerce PAM.

11.1 Descrição do processo de DevOps

Etapa	Nome	Descrição
1	Demanda e planejamento	A equipe define requisitos, funcionalidades e divisão de tarefas.
2	Codificação	Desenvolvimento das telas, API e modelos de dados.
3	Versionamento	Envio das alterações para o repositório Git/GitHub.
4	Integração contínua	Execução automática de instalação, lint, testes e build.
5	Testes	Verificação de rotas, login, cadastro, CRUD e integração com banco.
6	Deploy	Publicação do backend, frontend e configuração do MongoDB Atlas.
7	Monitoramento	Acompanhamento de logs, status da API, erros e disponibilidade.
8	Feedback	Correção de problemas e melhoria contínua.

11.2 Ferramentas sugeridas

Para desenhar o fluxograma podem ser utilizados Draw.io, Miro, Lucidchart, Canva, OpenOffice Draw ou SmartArt do Word. Para automação e testes, podem ser utilizados GitHub Actions, Robot Framework, Selenium IDE, Power Automate ou outras ferramentas equivalentes, de acordo com a infraestrutura escolhida pela equipe.

12 COMO EXECUTAR O PROJETO

12.1 Backend

- Acessar a pasta backend.
- Executar `npm install` caso as dependências não estejam instaladas.
- Criar ou conferir o arquivo `.env` com `MONGO_URI` e `PORT`.
- Executar `npm start`.
- Verificar a mensagem de conexão ao MongoDB Atlas e o servidor em `http://localhost:5000`.

12.2 Frontend

- Acessar a pasta frontend.
- Executar `npm install` caso as dependências não estejam instaladas.
- Executar `npm run dev`.
- Abrir o endereço exibido pelo Vite, geralmente `http://localhost:5173`.
- Testar cadastro, login, home da loja e painel administrativo.

12.3 Observação de segurança

O arquivo `.env` real não deve ser anexado em documentação nem enviado para repositórios públicos, pois pode conter credenciais do MongoDB Atlas. O ideal é manter apenas um arquivo `.env.example` com nomes de variáveis e valores fictícios.

13 ANÁLISE FINAL: O QUE FALTA OU PODE MELHORAR

A análise do projeto mostra que a base principal do e-commerce foi desenvolvida e que a documentação técnica e o fluxograma de DevOps foram preparados neste documento. Mesmo assim, há pontos que podem ser ajustados para aumentar a qualidade da entrega e reduzir riscos de erro na apresentação.

Ponto verificado	Situação	Recomendação
Fluxograma de DevOps	Atendido no documento	Se o professor pedir arquivo separado, exportar também em PNG/PDF pelo Draw.io, Canva ou Word.
Testes automatizados	Falta no código	Adicionar scripts e ferramentas de teste automatizado se houver tempo.
Monitoramento real	Falta no código	Inserir logs estruturados, rota de health check mais completa e ferramenta de monitoramento.
Pipeline CI/CD	Falta no código	Criar workflow no GitHub Actions para instalar, testar e fazer build.

Proteção do painel ADM	Falta parcialmente	As rotas administrativas não possuem middleware de autenticação/autorização.
Carrinho e pedido	Parcial	O carrinho mostra contador fixo e a compra é simulada por alert, sem pedido salvo no banco.
Importações com diferença de maiúsculas/minúsculas	Precisa revisar	App.jsx importa alguns arquivos em caixa baixa; em Linux/hospedagem pode falhar.
Arquivo .env no ZIP	Precisa remover antes de publicar	Não divulgar credenciais; usar .env.example.
Projeto mobile	Atenção	A disciplina é PAM, porém a atividade e o ZIP estão como MERN Web. Se for exigido app mobile, adaptar para React Native/Expo ou justificar como aplicação web responsiva.

Conclusão: o trabalho atende ao núcleo da atividade, pois apresenta um software de e-commerce em MERN, documentação técnica e fluxograma lógico de DevOps. Para uma entrega mais completa, recomenda-se corrigir as importações sensíveis a maiúsculas/minúsculas, retirar credenciais do ZIP, criar um repositório no GitHub, adicionar testes automatizados simples e configurar um pipeline básico de CI/CD.

14 CONCLUSÃO

O sistema desenvolvido representa uma solução de e-commerce com arquitetura MERN, composta por interface React, API Node.js/Express e banco de dados MongoDB Atlas. A aplicação demonstra conceitos importantes de desenvolvimento full stack, como separação de camadas, consumo de API, modelagem de dados, autenticação básica e operações CRUD.

A documentação apresenta a análise da atividade, o levantamento técnico, a arquitetura, as rotas, os modelos, os testes sugeridos e o fluxo lógico de DevOps. Dessa forma, o material final organiza a entrega da equipe e evidencia como a fábrica de software poderia conduzir o desenvolvimento, a automação, os testes, o deploy e o monitoramento da solução.

REFERÊNCIAS

PISTILLE, Benir Falcão. Atividade Arquitetura e Desenvolvimento MERN Full Stack Web. Material de aula, 2026.

Equipe do projeto. Código-fonte do sistema PAM - E-commerce. Arquivo ZIP analisado, 2026.

APÊNDICE A - CÓDIGO-FONTE PRINCIPAL DA SOLUÇÃO

A seguir, são apresentados os principais arquivos de codificação do projeto. A pasta `node_modules`, arquivos `package-lock` e o arquivo `.env` real não foram reproduzidos integralmente por não fazerem parte da codificação autoral da equipe ou por conterem dados sensíveis. Para entrega pública, recomenda-se substituir `.env` por `.env.example`.

A.1 backend/package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node server.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^16.6.1",
    "express": "^5.2.1",
    "mongoose": "^9.6.2"
  }
}
```

A.2 backend/server.js

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const usuariosRouter = require('./routes/usuarios');
const produtosRouter = require('./routes/produtos');

const app = express();
const PORT = process.env.PORT || 5000;

// Middlewares
app.use(cors());
app.use(express.json());

// Rotas da API
app.use('/api/usuarios', usuariosRouter);
app.use('/api/produtos', produtosRouter);

// Rota de saúde para testar se o servidor está online
app.get('/', (req, res) => {
  res.json({ status: 'PAM API online [START]', versao: '1.0.0' });
});

// Conexão com MongoDB Atlas e inicialização do servidor
mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log('[OK] Conectado ao MongoDB Atlas com sucesso!');
    app.listen(PORT, () => {
      console.log(`[START] Servidor rodando em http://localhost:${PORT}`);
    });
  })
  .catch((err) => console.error('Erro ao conectar ao MongoDB:', err));
```

```

    .catch((err) => {
      console.error('[ERRO] Erro ao conectar ao MongoDB:', err.message);
      process.exit(1);
    });

```

A.3 backend/models/Usuario.js

```

const mongoose = require('mongoose');

const usuarioSchema = new mongoose.Schema({
  nome: {
    type: String,
    required: [true, 'Nome é obrigatório'],
    trim: true
  },
  usuario: {
    type: String,
    required: [true, 'Nome de usuário é obrigatório'],
    unique: true,
    trim: true,
    lowercase: true
  },
  email: {
    type: String,
    required: [true, 'E-mail é obrigatório'],
    unique: true,
    trim: true,
    lowercase: true
  },
  senha: {
    type: String,
    required: [true, 'Senha é obrigatória'],
    minlength: 6
  },
  endereco: {
    type: String,
    default: ''
  },
  telefone: {
    type: String,
    default: ''
  },
  cep: {
    type: String,
    default: ''
  },
  role: {
    type: String,
    enum: ['comum', 'admin'],
    default: 'comum'
  }
}, { timestamps: true });

module.exports = mongoose.model('Usuario', usuarioSchema);

```

A.4 backend/models/Produto.js

```

const mongoose = require('mongoose');

const produtoSchema = new mongoose.Schema({
  nome: {
    type: String,
    required: [true, 'Nome do produto é obrigatório'],
    trim: true
  },
  descricao: {
    type: String,
    default: 'Sem descrição disponível.'
  },
  preco: {
    type: Number,
    required: [true, 'Preço é obrigatório'],
    min: [0, 'Preço não pode ser negativo']
  },

```

```

    imagem: {
      type: String,
      default: 'https://placeholder.co/400x400/1a1a2e/ffffff?text=Produto'
    },
    estoque: {
      type: Number,
      default: 0,
      min: 0
    }
  }, { timestamps: true });

module.exports = mongoose.model('Produto', produtoSchema);

```

A.5 backend/routes/usuarios.js

```

const express = require('express');
const router = express.Router();
const bcrypt = require('bcryptjs');
const Usuario = require('../models/Usuario');

// POST /api/usuarios/cadastro - Criar novo usuário
router.post('/cadastro', async (req, res) => {
  try {
    const { nome, usuario, email, senha, confirmacao, endereco, telefone, cep } = req.body;

    if (senha !== confirmacao) {
      return res.status(400).json({ erro: 'As senhas não coincidem.' });
    }

    const emailExistente = await Usuario.findOne({ email });
    if (emailExistente) {
      return res.status(400).json({ erro: 'Este e-mail já está cadastrado.' });
    }

    const usuarioExistente = await Usuario.findOne({ usuario });
    if (usuarioExistente) {
      return res.status(400).json({ erro: 'Este nome de usuário já está em uso.' });
    }

    const salt = await bcrypt.genSalt(10);
    const senhaHash = await bcrypt.hash(senha, salt);

    const novoUsuario = new Usuario({
      nome,
      usuario,
      email,
      senha: senhaHash,
      endereco,
      telefone,
      cep
    });

    await novoUsuario.save();

    res.status(201).json({
      mensagem: 'Usuário cadastrado com sucesso!',
      usuario: {
        id: novoUsuario._id,
        nome: novoUsuario.nome,
        usuario: novoUsuario.usuario,
        email: novoUsuario.email,
        role: novoUsuario.role
      }
    });
  } catch (err) {
    console.error('Erro no cadastro:', err);
    res.status(500).json({ erro: 'Erro interno do servidor.' });
  }
});

// POST /api/usuarios/login - Autenticar usuário

```

```

router.post('/login', async (req, res) => {
  try {
    const { email, senha } = req.body;

    const usuario = await Usuario.findOne({ email });
    if (!usuario) {
      return res.status(401).json({ erro: 'E-mail ou senha incorretos.' });
    }

    const senhaCorreta = await bcrypt.compare(senha, usuario.senha);
    if (!senhaCorreta) {
      return res.status(401).json({ erro: 'E-mail ou senha incorretos.' });
    }

    res.json({
      mensagem: 'Login realizado com sucesso!',
      usuario: {
        id: usuario._id,
        nome: usuario.nome,
        usuario: usuario.usuario,
        email: usuario.email,
        role: usuario.role
      }
    });
  } catch (err) {
    console.error('Erro no login:', err);
    res.status(500).json({ erro: 'Erro interno do servidor.' });
  }
});

// GET /api/usuarios - Listar todos os usuários (admin)
router.get('/', async (req, res) => {
  try {
    const usuarios = await Usuario.find().select('-senha').sort({ createdAt: -1 });
    res.json(usuarios);
  } catch (err) {
    res.status(500).json({ erro: 'Erro ao buscar usuários.' });
  }
});

// GET /api/usuarios/:id - Buscar um usuário por ID
router.get('/:id', async (req, res) => {
  try {
    const usuario = await Usuario.findById(req.params.id).select('-senha');
    if (!usuario) return res.status(404).json({ erro: 'Usuário não encontrado.' });
    res.json(usuario);
  } catch (err) {
    res.status(500).json({ erro: 'Erro ao buscar usuário.' });
  }
});

// PUT /api/usuarios/:id - Atualizar perfil
router.put('/:id', async (req, res) => {
  try {
    const { nome, usuario, email, endereco, telefone, cep } = req.body;

    const atualizado = await Usuario.findByIdAndUpdate(
      req.params.id,
      { nome, usuario, email, endereco, telefone, cep },
      { new: true, runValidators: true }
    ).select('-senha');

    if (!atualizado) return res.status(404).json({ erro: 'Usuário não encontrado.' });
    res.json({ mensagem: 'Perfil atualizado com sucesso!', usuario: atualizado });
  } catch (err) {
    res.status(500).json({ erro: 'Erro ao atualizar usuário.' });
  }
});

// DELETE /api/usuarios/:id - Deletar usuário (admin)
router.delete('/:id', async (req, res) => {
  try {

```

```

    const deletado = await Usuario.findByIdAndDelete(req.params.id);
    if (!deletado) return res.status(404).json({ erro: 'Usuário não encontrado.' });
    res.json({ mensagem: 'Usuário deletado com sucesso.' });
  } catch (err) {
    res.status(500).json({ erro: 'Erro ao deletar usuário.' });
  }
});

```

```
module.exports = router;
```

A.6 backend/routes/produtos.js

```

const express = require('express');
const router = express.Router();
const Produto = require('../models/Produto');

// GET /api/produtos - Listar todos os produtos
router.get('/', async (req, res) => {
  try {
    const produtos = await Produto.find().sort({ createdAt: -1 });
    res.json(produtos);
  } catch (err) {
    res.status(500).json({ erro: 'Erro ao buscar produtos.' });
  }
});

// GET /api/produtos/:id - Buscar produto por ID
router.get('/:id', async (req, res) => {
  try {
    const produto = await Produto.findById(req.params.id);
    if (!produto) return res.status(404).json({ erro: 'Produto não encontrado.' });
    res.json(produto);
  } catch (err) {
    res.status(500).json({ erro: 'Erro ao buscar produto.' });
  }
});

// POST /api/produtos - Criar novo produto (admin)
router.post('/', async (req, res) => {
  try {
    const { nome, descricao, preco, imagem, estoque } = req.body;

    const novoProduto = new Produto({ nome, descricao, preco, imagem, estoque });
    await novoProduto.save();

    res.status(201).json({ mensagem: 'Produto criado com sucesso!', produto: novoProduto });
  } catch (err) {
    console.error('Erro ao criar produto:', err);
    res.status(500).json({ erro: 'Erro ao criar produto.' });
  }
});

// PUT /api/produtos/:id - Editar produto (admin)
router.put('/:id', async (req, res) => {
  try {
    const { nome, descricao, preco, imagem, estoque } = req.body;

    const atualizado = await Produto.findByIdAndUpdate(
      req.params.id,
      { nome, descricao, preco, imagem, estoque },
      { new: true, runValidators: true }
    );

    if (!atualizado) return res.status(404).json({ erro: 'Produto não encontrado.' });
    res.json({ mensagem: 'Produto atualizado com sucesso!', produto: atualizado });
  } catch (err) {
    res.status(500).json({ erro: 'Erro ao atualizar produto.' });
  }
});

// DELETE /api/produtos/:id - Deletar produto (admin)

```

```

router.delete('/:id', async (req, res) => {
  try {
    const deletado = await Produto.findByIdAndDelete(req.params.id);
    if (!deletado) return res.status(404).json({ erro: 'Produto não encontrado.' });
    res.json({ mensagem: 'Produto deletado com sucesso.' });
  } catch (err) {
    res.status(500).json({ erro: 'Erro ao deletar produto.' });
  }
});

module.exports = router;

```

A.7 frontend/package.json

```

{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "lucide-react": "^1.16.0",
    "react": "^19.2.6",
    "react-dom": "^19.2.6",
    "react-router-dom": "^7.15.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^10.3.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.6.0",
    "vite": "^8.0.12"
  }
}

```

A.8 frontend/src/main.jsx

```

import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './App.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)

```

A.9 frontend/src/App.jsx

```

import { Routes, Route } from 'react-router-dom'
import Header from './components/header'
import PaginaInicial from './pages/PaginaInicial'
import Home from './pages/home'
import Login from './pages/login'
import Cadastro from './pages/Cadastro'
import ProdutoDetalhe from './pages/produtodetalhe'
import PainelAdm from './pages/paineladm'
import Perfil from './pages/Perfil'

function App() {

```

```

return (
  <div className="app-container">
    <Header />
    <main className="main-content">
      <Routes>
        <Route path="/" element={<PaginaInicial />} />
        <Route path="/home" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/cadastro" element={<Cadastro />} />
        <Route path="/perfil" element={<Perfil />} />
        <Route path="/produto/:id" element={<ProdutoDetalhe />} />
        <Route path="/adm" element={<PainelAdm />} />
      </Routes>
    </main>
  </div>
)
}

export default App

```

A.10 frontend/src/services/api.js

```

const API_URL = 'http://localhost:5000/api';

// Helper genérico para fazer requisições
async function request(endpoint, options = {}) {
  const response = await fetch(`${API_URL}${endpoint}`, {
    headers: { 'Content-Type': 'application/json' },
    ...options
  });
  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.erro || 'Erro na requisição');
  }
  return data;
}

// ===== USUÁRIOS =====
export const usuariosAPI = {
  cadastrar: (dados) => request('/usuarios/cadastro', {
    method: 'POST',
    body: JSON.stringify(dados)
  }),

  login: (dados) => request('/usuarios/login', {
    method: 'POST',
    body: JSON.stringify(dados)
  }),

  listar: () => request('/usuarios'),

  buscarPorId: (id) => request(`/usuarios/${id}`),

  atualizar: (id, dados) => request(`/usuarios/${id}`, {
    method: 'PUT',
    body: JSON.stringify(dados)
  }),

  deletar: (id) => request(`/usuarios/${id}`, { method: 'DELETE' })
};

// ===== PRODUTOS =====
export const produtosAPI = {
  listar: () => request('/produtos'),

  buscarPorId: (id) => request(`/produtos/${id}`),

  criar: (dados) => request('/produtos', {
    method: 'POST',
    body: JSON.stringify(dados)
  })
}

```

```

    }},

    atualizar: (id, dados) => request(`/produtos/${id}`, {
      method: 'PUT',
      body: JSON.stringify(dados)
    }),

    deletar: (id) => request(`/produtos/${id}`, { method: 'DELETE' })
  };

```

A.11 frontend/src/components/Header.jsx

```

import { Link } from 'react-router-dom'
import { ShoppingCart, LogIn, User } from 'lucide-react'
import './Header.css'

function Header() {
  return (
    <header className="header">
      <div className="header-content">
        <div className="logo-section">
          <Link to="/" className="logo">PAM - E-commerce</Link>
        </div>

        <nav className="nav-links">
          <Link to="/home" className="nav-item">Loja (Home)</Link>

          <div className="cart-icon">
            <ShoppingCart size={24} />
            <span className="cart-count">0</span>
          </div>

          <Link to="/perfil" className="nav-item" title="Meu Perfil">
            <User size={24} />
          </Link>

          <Link to="/login" className="login-btn">
            <LogIn size={20} />
            Login
          </Link>
        </nav>
      </div>
    </header>
  )
}

export default Header

```

A.12 frontend/src/components/Header.css

```

.header {
  background-color: #2c3e50;
  color: white;
  padding: 1rem 2rem;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.header-content {
  max-width: 1200px;
  margin: 0 auto;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.logo {
  color: white;
  text-decoration: none;
  font-size: 1.5rem;
  font-weight: bold;
}

```

```

}

.nav-links {
  display: flex;
  align-items: center;
  gap: 2rem;
}

.nav-item {
  color: #ecf0f1;
  text-decoration: none;
  font-size: 1.1rem;
}

.nav-item:hover {
  color: #3498db;
}

.cart-icon {
  position: relative;
  cursor: pointer;
  display: flex;
  align-items: center;
}

.cart-count {
  position: absolute;
  top: -8px;
  right: -10px;
  background-color: #e74c3c;
  color: white;
  border-radius: 50%;
  padding: 2px 6px;
  font-size: 0.8rem;
  font-weight: bold;
}

.login-btn {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  background-color: #3498db;
  color: white;
  text-decoration: none;
  padding: 0.5rem 1rem;
  border-radius: 4px;
  font-weight: bold;
  transition: background 0.3s;
}

.login-btn:hover {
  background-color: #2980b9;
}

```

A.13 frontend/src/pages/PaginaInicial.jsx

```

import { Link } from 'react-router-dom'

function PaginaInicial() {
  return (
    <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', marginTop: '4rem' }}>
      <section style={{ background: 'white', padding: '3rem', borderRadius: '8px',
        boxShadow: '0 4px 6px rgba(0,0,0,0.1)', textAlign: 'center', maxWidth: '600px' }}>
        <h1 style={{ color: '#2c3e50', marginBottom: '1rem' }}>Matéria: PAM</h1>
        <h2 style={{ color: '#3498db', marginBottom: '2rem' }}>Atividade 2º Bimestre</h2>

        <div style={{ textAlign: 'left', marginBottom: '2rem', padding: '1rem', background: '#f8f9fa', borderRadius: '8px' }}>
          <h3 style={{ marginBottom: '1rem' }}>Integrantes do Grupo:</h3>

```

```

        <ul style={{ listStyleType: 'none', padding: 0, fontSize: '1.1rem', lineHeight:
'1.8' }}>
            <li>- Alan Marcelo</li>
            <li>- Benicio Rodrigues</li>
            <li>- Henrique Furquim</li>
            <li>- Thalles Yuji</li>
        </ul>
    </div>

    <Link to="/home" style={{ display: 'inline-block', padding: '1rem 2rem', background:
'#27ae60', color: 'white', textDecoration: 'none', borderRadius: '4px', fontWeight: 'bold',
fontSize: '1.2rem' }}>
        Acessar a Loja
    </Link>
</section>
</div>
)
}

export default PaginaInicial

```

A.14 frontend/src/pages/Home.jsx

```

import { useState, useEffect } from 'react'
import { Link } from 'react-router-dom'
import { produtosAPI } from '../services/api'
import './Home.css'

function Home() {
    const [produtos, setProdutos] = useState([])
    const [carregando, setCarregando] = useState(true)
    const [erro, setErro] = useState('')

    useEffect(() => {
        const buscarProdutos = async () => {
            try {
                const dados = await produtosAPI.listar()
                setProdutos(dados)
            } catch (err) {
                setErro('Não foi possível carregar os produtos. Verifique se o servidor está
rodando.')
            } finally {
                setCarregando(false)
            }
        }
        buscarProdutos()
    }, [])

    if (carregando) {
        return (
            <div className="home-container">
                <p style={{ textAlign: 'center', marginTop: '3rem', fontSize: '1.2rem', color:
'#666' }}>
                    [carregando] Carregando produtos...
                </p>
            </div>
        )
    }

    if (erro) {
        return (
            <div className="home-container">
                <div style={{ textAlign: 'center', marginTop: '3rem', color: '#e74c3c', background:
'#fde8e8', padding: '1.5rem', borderRadius: '8px' }}>
                    <p>[aviso] {erro}</p>
                </div>
            </div>
        )
    }

    return (
        <div className="home-container">

```

```

<section className="produtos-vitrine">
  <h3>Nossos Produtos</h3>
  {produtos.length === 0 ? (
    <div style={{ textAlign: 'center', padding: '3rem', color: '#666' }}>
      <p>Nenhum produto cadastrado ainda.</p>
      <Link to="/adm" style={{ color: '#3498db' }}>Adicionar produtos no painel
admin</Link>
    </div>
  ) : (
    <div className="grid-produtos">
      {produtos.map(produto => (
        <Link to={`/produto/${produto._id}`} key={produto._id} className="card-
produto">
          <img src={produto.imagem} alt={produto.nome} />
          <h4>{produto.nome}</h4>
          <p className="preco">R$ {produto.preco.toFixed(2)}</p>
        </Link>
      ))}
    </div>
  )}
</section>
</div>
)
}

export default Home

```

A.15 frontend/src/pages/Home.css

```

.grupo-info {
  background-color: white;
  padding: 1.5rem;
  border-radius: 8px;
  margin-bottom: 2rem;
  box-shadow: 0 2px 5px rgba(0,0,0,0.05);
  border-left: 5px solid #3498db;
}

.grupo-info h2 {
  color: #2c3e50;
  margin-bottom: 0.5rem;
}

.produtos-vitrine h3 {
  font-size: 1.8rem;
  margin-bottom: 1.5rem;
  color: #2c3e50;
}

.grid-produtos {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 2rem;
}

.card-produto {
  background: white;
  border-radius: 8px;
  padding: 1rem;
  text-align: center;
  text-decoration: none;
  color: inherit;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  transition: transform 0.2s, box-shadow 0.2s;
}

.card-produto:hover {
  transform: translateY(-5px);
  box-shadow: 0 6px 12px rgba(0,0,0,0.15);
}

.card-produto img {

```

```

width: 100%;
border-radius: 4px;
margin-bottom: 1rem;
}

.card-produto h4 {
font-size: 1.1rem;
margin-bottom: 0.5rem;
}

.card-produto .preco {
font-size: 1.25rem;
font-weight: bold;
color: #27ae60;
}

```

A.16 frontend/src/pages/Login.jsx

```

import { useState } from 'react'
import { Link, useNavigate } from 'react-router-dom'
import { usuariosAPI } from '../services/api'

function Login() {
  const [email, setEmail] = useState('')
  const [senha, setSenha] = useState('')
  const [erro, setErro] = useState('')
  const [carregando, setCarregando] = useState(false)
  const navigate = useNavigate()

  const handleLogin = async (e) => {
    e.preventDefault()
    setErro('')
    setCarregando(true)

    try {
      const resposta = await usuariosAPI.login({ email, senha })
      // Salva o usuário na sessão do navegador
      sessionStorage.setItem('usuario', JSON.stringify(resposta.usuario))
      alert(`Bem-vindo(a), ${resposta.usuario.nome}!`)
      navigate('/home')
    } catch (err) {
      setErro(err.message)
    } finally {
      setCarregando(false)
    }
  }

  return (
    <div style={{ display: 'flex', justifyContent: 'center', marginTop: '4rem' }}>
      <div style={{ background: 'white', padding: '2rem', borderRadius: '8px', width: '400px', boxShadow: '0 4px 6px rgba(0,0,0,0.1)' }}>
        <h2 style={{ textAlign: 'center', marginBottom: '1.5rem', color: '#2c3e50' }}>Acesse sua Conta</h2>

        {erro && (
          <div style={{ background: '#fde8e8', color: '#e74c3c', padding: '0.8rem', borderRadius: '4px', marginBottom: '1rem', textAlign: 'center' }}>
            {erro}
          </div>
        )}

        <form onSubmit={handleLogin} style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>
          <div>
            <label>E-mail</label>
            <input required type="email" style={inputStyle} placeholder="exemplo@email.com" value={email} onChange={(e) => setEmail(e.target.value)} />
          </div>
          <div>
            <label>Senha</label>
            <input required type="password" style={inputStyle} placeholder="Sua senha" value={senha} onChange={(e) => setSenha(e.target.value)} />

```

```

    </div>

    <button type="submit" disabled={carregando} style={{ width: '100%', padding:
'1rem', background: carregando ? '#95a5a6' : '#3498db', color: 'white', border: 'none',
borderRadius: '4px', fontSize: '1.1rem', cursor: carregando ? 'not-allowed' : 'pointer',
marginTop: '1rem' }}>
      {carregando ? 'Entrando...' : 'Entrar'}
    </button>
  </form>

  <p style={{ textAlign: 'center', marginTop: '1rem' }}>
    Não tem uma conta? <Link to="/cadastro" style={{ color: '#3498db' }}>Crie
    agora</Link>
  </p>

  <div style={{ marginTop: '2rem', borderTop: '1px solid #eee', paddingTop: '1rem',
  textAlign: 'center' }}>
    <p style={{ fontSize: '0.8rem', color: '#666', marginBottom: '0.5rem' }}>Acesso
    Rápido para Desenvolvimento:</p>
    <Link to="/adm" style={{ fontSize: '0.9rem', color: '#e74c3c', textDecoration:
'none' }}>Ir para o Painel do Administrador</Link>
  </div>
</div>
)
}

const inputStyle = { width: '100%', padding: '0.8rem', borderRadius: '4px', border: '1px
solid #ccc', marginTop: '0.3rem' }
export default Login

```

A.17 frontend/src/pages/Cadastro.jsx

```

import { useState } from 'react'
import { Link, useNavigate } from 'react-router-dom'
import { usuariosAPI } from '../services/api'

function Cadastro() {
  const [formData, setFormData] = useState({
    nome: '', usuario: '', endereco: '', telefone: '', email: '', cep: '', senha: '',
    confirmacao: ''
  })
  const [erro, setErro] = useState('')
  const [carregando, setCarregando] = useState(false)
  const navigate = useNavigate()

  const handleTelefone = (e) => {
    let value = e.target.value.replace(/\D/g, '')
    if (value.length > 11) value = value.slice(0, 11)
    if (value.length > 2) value = `(${value.slice(0, 2)}) ${value.slice(2)}`
    if (value.length > 10) value = `${value.slice(0, 10)}-${value.slice(10)}`
    setFormData({ ...formData, telefone: value })
  }

  const handleCEP = (e) => {
    let value = e.target.value.replace(/\D/g, '')
    if (value.length > 8) value = value.slice(0, 8)
    if (value.length > 5) value = `${value.slice(0, 5)}-${value.slice(5)}`
    setFormData({ ...formData, cep: value })
  }

  const handleSubmit = async (e) => {
    e.preventDefault()
    setErro('')

    if (formData.senha !== formData.confirmacao) {
      setErro('As senhas não coincidem!')
      return
    }

    setCarregando(true)

```

```

    try {
      await usuariosAPI.cadastrar(formData)
      alert('Cadastro realizado com sucesso! Faça login para continuar.')
      navigate('/login')
    } catch (err) {
      setErro(err.message)
    } finally {
      setCarregando(false)
    }
  }

  return (
    <div style={{ display: 'flex', justifyContent: 'center', marginTop: '2rem',
marginBottom: '2rem' }}>
      <div style={{ background: 'white', padding: '2rem', borderRadius: '8px', width:
'500px', boxShadow: '0 4px 6px rgba(0,0,0,0.1)' }}>
        <h2 style={{ textAlign: 'center', marginBottom: '1.5rem', color: '#2c3e50' }}>Criar
Nova Conta</h2>

        {erro && (
          <div style={{ background: '#fde8e8', color: '#e74c3c', padding: '0.8rem',
borderRadius: '4px', marginBottom: '1rem', textAlign: 'center' }}>
            {erro}
          </div>
        )}

        <form onSubmit={handleSubmit} style={{ display: 'flex', flexDirection: 'column',
gap: '1rem' }}>
          <div>
            <label>Nome Completo</label>
            <input required type="text" style={inputStyle} value={formData.nome}
onChange={e => setFormData({...formData, nome: e.target.value})} />
          </div>
          <div>
            <label>Nome de Usuário</label>
            <input required type="text" style={inputStyle} value={formData.usuario}
onChange={e => setFormData({...formData, usuario: e.target.value})} />
          </div>
          <div>
            <label>Endereço</label>
            <input required type="text" style={inputStyle} value={formData.endereco}
onChange={e => setFormData({...formData, endereco: e.target.value})} />
          </div>
          <div style={{ display: 'flex', gap: '1rem' }}>
            <div style={{ flex: 1 }}>
              <label>CEP</label>
              <input required type="text" placeholder="XXXXX-XXX" style={inputStyle}
value={formData.cep} onChange={handleCEP} />
            </div>
            <div style={{ flex: 1 }}>
              <label>Telefone</label>
              <input required type="text" placeholder="(XX) XXXXX-XXXX" style={inputStyle}
value={formData.telefone} onChange={handleTelefone} />
            </div>
          </div>
          <div>
            <label>E-mail</label>
            <input required type="email" placeholder="exemplo@email.com" style={inputStyle}
value={formData.email} onChange={e => setFormData({...formData, email: e.target.value})}
/>
          </div>
          <div style={{ display: 'flex', gap: '1rem' }}>
            <div style={{ flex: 1 }}>
              <label>Senha</label>
              <input required type="password" style={inputStyle} value={formData.senha}
onChange={e => setFormData({...formData, senha: e.target.value})} />
            </div>
            <div style={{ flex: 1 }}>
              <label>Confirmar Senha</label>
              <input required type="password" style={inputStyle}
value={formData.confirmacao} onChange={e => setFormData({...formData, confirmacao:
e.target.value})} />
            </div>
          </div>
        </form>
      </div>
    </div>
  )

```

```

        <button type="submit" disabled={carregando} style={{ padding: '1rem', background:
carregando ? '#95a5a6' : '#27ae60', color: 'white', border: 'none', borderRadius: '4px',
cursor: carregando ? 'not-allowed' : 'pointer', marginTop: '1rem', fontWeight: 'bold' }}>
        {carregando ? 'Cadastrando...' : 'Cadastrar'}
    </button>
</form>
    <p style={{ textAlign: 'center', marginTop: '1rem' }}>
        Já possui conta? <Link to="/login" style={{ color: '#3498db' }}>Faça login</Link>
    </p>
</div>
</div>
)
}

const inputStyle = { width: '100%', padding: '0.8rem', borderRadius: '4px', border: '1px
solid #ccc', marginTop: '0.3rem' }
export default Cadastro

```

A.18 frontend/src/pages/Perfil.jsx

```

import { useState, useEffect } from 'react'
import { Camera } from 'lucide-react'
import { usuariosAPI } from '../services/api'

function Perfil() {
    const [perfil, setPerfil] = useState(null)
    const [carregando, setCarregando] = useState(true)
    const [salvando, setSalvando] = useState(false)
    const [mensagem, setMensagem] = useState('')

    useEffect(() => {
        // Tenta pegar o usuário logado da sessão
        const usuarioSessao = sessionStorage.getItem('usuario')
        if (usuarioSessao) {
            const usuario = JSON.parse(usuarioSessao)
            // Busca os dados atualizados do servidor
            usuariosAPI.buscarPorId(usuario.id)
                .then(dados => setPerfil(dados))
                .catch(() => setPerfil(usuario))
                .finally(() => setCarregando(false))
        } else {
            // Sem sessão: usa dados demonstrativos
            setPerfil({
                nome: 'Thalles Yuji', usuario: 'thallesy', endereco: 'Rua Exemplo, 123',
                telefone: '(11) 99999-9999', email: 'thalles@email.com', cep: '01000-000'
            })
            setCarregando(false)
        }
    }, [])

    const handleSubmit = async (e) => {
        e.preventDefault()
        setMensagem('')
        setSalvando(true)

        try {
            const usuarioSessao = sessionStorage.getItem('usuario')
            if (usuarioSessao) {
                const usuario = JSON.parse(usuarioSessao)
                const atualizado = await usuariosAPI.atualizar(usuario.id, perfil)
                sessionStorage.setItem('usuario', JSON.stringify(atualizado.usuario))
            }
            setMensagem('[OK] Dados atualizados com sucesso!')
        } catch (err) {
            setMensagem('[ERRO] Erro ao atualizar: ' + err.message)
        } finally {
            setSalvando(false)
        }
    }

    if (carregando) return <div style={{ textAlign: 'center', marginTop: '3rem' }}>[carregando] Carregando perfil...</div>
}

```

```

    return (
      <div style={{ display: 'flex', justifyContent: 'center', marginTop: '2rem' }}>
        <div style={{ background: 'white', padding: '2rem', borderRadius: '8px', width:
'600px', boxShadow: '0 4px 6px rgba(0,0,0,0.1)' }}>
          <h2 style={{ marginBottom: '1.5rem', color: '#2c3e50', borderBottom: '2px solid
#eee', paddingBottom: '0.5rem' }}>Meu Perfil</h2>

          {mensagem && (
            <div style={{ padding: '0.8rem', borderRadius: '4px', marginBottom: '1rem',
background: mensagem.startsWith('[OK]') ? '#e8fde8' : '#fde8e8', color:
mensagem.startsWith('[OK]') ? '#27ae60' : '#e74c3c', textAlign: 'center' }}>
              {mensagem}
            </div>
          )}

          <div style={{ display: 'flex', gap: '2rem' }}>
            {/* Foto */}
            <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center' }}>
              <div style={{ width: '150px', height: '150px', background: '#eee', borderRadius:
'50%', display: 'flex', justifyContent: 'center', alignItems: 'center', overflow: 'hidden',
marginBottom: '1rem' }}>
                <Camera size={40} color="#ccc" />
              </div>
              <button style={{ padding: '0.5rem 1rem', background: '#3498db', color: 'white',
border: 'none', borderRadius: '4px', cursor: 'pointer' }}>
                Alterar Foto
              </button>
            </div>

            {/* Formulário */}
            <form onSubmit={handleSubmit} style={{ flex: 1, display: 'flex', flexDirection:
'column', gap: '1rem' }}>
              <div>
                <label>Nome Completo</label>
                <input type="text" style={inputStyle} value={perfil?.nome || ''} onChange={(e)
=> setPerfil({...perfil, nome: e.target.value})} />
              </div>
              <div>
                <label>Nome de Usuário</label>
                <input type="text" style={inputStyle} value={perfil?.usuario || ''}
onChange={(e) => setPerfil({...perfil, usuario: e.target.value})} />
              </div>
              <div>
                <label>E-mail</label>
                <input type="email" style={inputStyle} value={perfil?.email || ''}
onChange={(e) => setPerfil({...perfil, email: e.target.value})} />
              </div>
              <div>
                <label style={{ color: '#7f8c8d' }}>Senha (Não é possível alterar por
aqui)</label>
                <input type="password" style={{ ...inputStyle, background: '#f8f9fa', cursor:
'not-allowed' }} value="*****" disabled />
              </div>
              <button type="submit" disabled={salvando} style={{ padding: '0.8rem',
background: salvando ? '#95a5a6' : '#2ecc71', color: 'white', border: 'none', borderRadius:
'4px', cursor: salvando ? 'not-allowed' : 'pointer', marginTop: '1rem', fontWeight: 'bold'
}}>
                {salvando ? 'Salvando...' : 'Salvar Alterações'}
              </button>
            </form>
          </div>
        </div>
      </div>
    )
  }

  const inputStyle = { width: '100%', padding: '0.6rem', borderRadius: '4px', border: '1px
solid #ccc', marginTop: '0.3rem' }
  export default Perfil

```

A.19 frontend/src/pages/ProdutoDetalhe.jsx

```
import { useState, useEffect } from 'react'
```

```

import { useParams, useNavigate } from 'react-router-dom'
import { produtosAPI } from '../services/api'

function ProdutoDetalhe() {
  const { id } = useParams()
  const navigate = useNavigate()
  const [produto, setProduto] = useState(null)
  const [carregando, setCarregando] = useState(true)
  const [erro, setErro] = useState('')

  useEffect(() => {
    const buscarProduto = async () => {
      try {
        const dados = await produtosAPI.buscarPorId(id)
        setProduto(dados)
      } catch (err) {
        setErro('Produto não encontrado.')
      } finally {
        setCarregando(false)
      }
    }
    buscarProduto()
  }, [id])

  const handleComprar = () => {
    alert('Processando a compra...')
    setTimeout(() => {
      alert('Compra bem sucedida! Obrigado pela preferência.')
      navigate('/home')
    }, 1000)
  }

  if (carregando) {
    return <div style={{ padding: '2rem', textAlign: 'center' }}>[carregando] Carregando
    produto...</div>
  }

  if (erro || !produto) {
    return <div style={{ padding: '2rem', textAlign: 'center', color: '#e74c3c' }}>[aviso]
    {erro}</div>
  }

  return (
    <div style={{ padding: '2rem', background: 'white', borderRadius: '8px' }}>
      <div style={{ display: 'flex', gap: '2rem' }}>
        <img src={produto.imagem} alt={produto.nome} style={{ borderRadius: '8px', width:
        '400px', height: '400px', objectFit: 'cover' }} />
        <div>
          <h1>{produto.nome}</h1>
          <p style={{ margin: '1rem 0', color: '#666', lineHeight: '1.6' }}>
            {produto.descricao}
          </p>
          {produto.estoque > 0 ? (
            <p style={{ color: '#27ae60', marginBottom: '0.5rem' }}>[OK] Em estoque
            ({produto.estoque} unidades)</p>
          ) : (
            <p style={{ color: '#e74c3c', marginBottom: '0.5rem' }}>[ERRO] Fora de
            estoque</p>
          )}
          <h2 style={{ color: '#27ae60', fontSize: '2rem', marginBottom: '2rem' }}>
            R$ {produto.preco.toFixed(2)}
          </h2>

          <button
            onClick={handleComprar}
            disabled={produto.estoque === 0}
            style={{ padding: '1rem 2rem', background: produto.estoque > 0 ? '#3498db' :
            '#95a5a6', color: 'white', border: 'none', borderRadius: '4px', fontSize: '1.1rem', cursor:
            produto.estoque > 0 ? 'pointer' : 'not-allowed', fontWeight: 'bold' }}>
            {produto.estoque > 0 ? 'Adicionar ao Carrinho e Comprar' : 'Produto
            Indisponível'}
          </button>
        </div>
      </div>
    )
  )
}

```

```

    </div>
  </div>
)
}

```

```
export default ProdutoDetalhe
```

A.20 frontend/src/pages/PainelAdm.jsx

```

import { useState, useEffect } from 'react'
import { Trash2, Edit, X, Check } from 'lucide-react'
import { produtosAPI, usuariosAPI } from '../services/api'

function PainelAdm() {
  const [produtos, setProdutos] = useState([])
  const [usuarios, setUsuarios] = useState([])
  const [carregando, setCarregando] = useState(true)

  // Modal de novo produto
  const [modalProduto, setModalProduto] = useState(false)
  const [produtoForm, setProdutoForm] = useState({ nome: '', descricao: '', preco: '',
  imagem: '', estoque: '' })
  const [editandoId, setEditandoId] = useState(null)

  const carregarDados = async () => {
    try {
      const [p, u] = await Promise.all([produtosAPI.listar(), usuariosAPI.listar()])
      setProdutos(p)
      setUsuarios(u)
    } catch (err) {
      alert('Erro ao carregar dados: ' + err.message)
    } finally {
      setCarregando(false)
    }
  }

  useEffect(() => { carregarDados() }, [])

  const handleSalvarProduto = async (e) => {
    e.preventDefault()
    try {
      const dados = { ...produtoForm, preco: parseFloat(produtoForm.preco), estoque:
      parseInt(produtoForm.estoque) }
      if (editandoId) {
        await produtosAPI.atualizar(editandoId, dados)
        alert('Produto atualizado!')
      } else {
        await produtosAPI.criar(dados)
        alert('Produto criado!')
      }
      setModalProduto(false)
      setProdutoForm({ nome: '', descricao: '', preco: '', imagem: '', estoque: '' })
      setEditandoId(null)
      carregarDados()
    } catch (err) {
      alert('Erro: ' + err.message)
    }
  }

  const handleEditarProduto = (produto) => {
    setProdutoForm({
      nome: produto.nome,
      descricao: produto.descricao,
      preco: produto.preco,
      imagem: produto.imagem,
      estoque: produto.estoque
    })
    setEditandoId(produto._id)
    setModalProduto(true)
  }

  const handleDeletarProduto = async (id) => {

```

```

    if (!confirm('Deletar este produto?')) return
    try {
      await produtosAPI.deletar(id)
      carregarDados()
    } catch (err) {
      alert('Erro ao deletar: ' + err.message)
    }
  }
}

const handleDeletarUsuario = async (id) => {
  if (!confirm('Deletar este usuário?')) return
  try {
    await usuariosAPI.deletar(id)
    carregarDados()
  } catch (err) {
    alert('Erro ao deletar: ' + err.message)
  }
}

if (carregando) return <div style={{ textAlign: 'center', marginTop: '3rem' }}>[carregando] Carregando painel...</div>

return (
  <div>
    <h1 style={{ marginBottom: '2rem' }}>Painel do Administrador</h1>

    { /* Modal Produto */ }
    { modalProduto && (
      <div style={{ position: 'fixed', top: 0, left: 0, right: 0, bottom: 0, background:
        'rgba(0,0,0,0.5)', display: 'flex', alignItems: 'center', justifyContent: 'center', zIndex:
        1000 }}>
        <div style={{ background: 'white', padding: '2rem', borderRadius: '8px', width:
        '500px' }}>
          <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
            'center', marginBottom: '1rem' }}>
            <h3>{editandoId ? 'Editar Produto' : 'Novo Produto'}</h3>
            <button onClick={() => { setModalProduto(false); setEditandoId(null) }}
              style={{ background: 'none', border: 'none', cursor: 'pointer' }}>X size={20} /></button>
          </div>
          <form onSubmit={handleSalvarProduto} style={{ display: 'flex', flexDirection:
            'column', gap: '0.8rem' }}>
            <input required placeholder="Nome do produto" style={inputStyle}
              value={produtoForm.nome} onChange={(e) => setProdutoForm({...produtoForm, nome:
                e.target.value})} />
            <textarea placeholder="Descrição" style={{ ...inputStyle, height: '80px',
              resize: 'vertical' }} value={produtoForm.descricao} onChange={(e) =>
              setProdutoForm({...produtoForm, descricao: e.target.value})} />
            <input required type="number" step="0.01" placeholder="Preço (ex: 199.99)"
              style={inputStyle} value={produtoForm.preco} onChange={(e) =>
              setProdutoForm({...produtoForm, preco: e.target.value})} />
            <input placeholder="URL da imagem" style={inputStyle}
              value={produtoForm.imagem} onChange={(e) => setProdutoForm({...produtoForm, imagem:
                e.target.value})} />
            <input type="number" placeholder="Estoque" style={inputStyle}
              value={produtoForm.estoque} onChange={(e) => setProdutoForm({...produtoForm, estoque:
                e.target.value})} />
            <button type="submit" style={{ padding: '0.8rem', background: '#27ae60',
              color: 'white', border: 'none', borderRadius: '4px', cursor: 'pointer', fontWeight: 'bold'
            }}>
              <Check size={16} style={{ marginRight: '0.5rem' }} />
              {editandoId ? 'Salvar Alterações' : 'Criar Produto'}
            </button>
          </form>
        </div>
      </div>
    ) }

    { /* Tabela de Produtos */ }
    <section style={{ marginBottom: '3rem', background: 'white', padding: '1.5rem',
      borderRadius: '8px' }}>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
        'center', marginBottom: '1rem' }}>
        <h2>Gerenciar Produtos ({produtos.length})</h2>

```

```

<button onClick={() => { setProdutoForm({ '', descricao: '', preco: '',
imagem: '', estoque: '' }); setEditandoId(null); setModalProduto(true) }} style={{
background: '#2ecc71', color: 'white', padding: '0.5rem 1rem', border: 'none', borderRadius:
'4px', cursor: 'pointer' }}>
  + Adicionar Produto
</button>
</div>
<table style={{ width: '100%', borderCollapse: 'collapse' }}>
  <thead>
    <tr style={{ background: '#f8f9fa', textAlign: 'left' }}>
      <th style={th}>Nome</th>
      <th style={th}>Preço</th>
      <th style={th}>Estoque</th>
      <th style={th}>Ações</th>
    </tr>
  </thead>
  <tbody>
    {produtos.length === 0 ? (
      <tr><td colSpan={4} style={{ padding: '1rem', textAlign: 'center', color:
'#666' }}>Nenhum produto cadastrado</td></tr>
    ) : (
      produtos.map(p => (
        <tr key={p._id}>
          <td style={td}>{p.nome}</td>
          <td style={td}>R$ {p.preco.toFixed(2)}</td>
          <td style={td}>{p.estoque}</td>
          <td style={{ ...td, display: 'flex', gap: '0.5rem' }}>
            <button onClick={() => handleEditarProduto(p)} style={{ cursor:
'pointer', background: 'none', border: 'none', color: '#f39c12' }}><Edit
size={20}></button>
            <button onClick={() => handleDeletarProduto(p._id)} style={{ cursor:
'pointer', background: 'none', border: 'none', color: '#e74c3c' }}><Trash2
size={20}></button>
          </td>
        </tr>
      ))
    )}
  </tbody>
</table>
</section>

/* Tabela de Usuários */
<section style={{ background: 'white', padding: '1.5rem', borderRadius: '8px' }}>
  <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center', marginBottom: '1rem' }}>
    <h2>Gerenciar Usuários ({usuarios.length})</h2>
  </div>
  <table style={{ width: '100%', borderCollapse: 'collapse' }}>
    <thead>
      <tr style={{ background: '#f8f9fa', textAlign: 'left' }}>
        <th style={th}>Nome</th>
        <th style={th}>Usuário</th>
        <th style={th}>E-mail</th>
        <th style={th}>Permissão</th>
        <th style={th}>Ações</th>
      </tr>
    </thead>
    <tbody>
      {usuarios.length === 0 ? (
        <tr><td colSpan={5} style={{ padding: '1rem', textAlign: 'center', color:
'#666' }}>Nenhum usuário cadastrado</td></tr>
      ) : (
        usuarios.map(u => (
          <tr key={u._id}>
            <td style={td}>{u.nome}</td>
            <td style={td}>{u.usuario}</td>
            <td style={td}>{u.email}</td>
            <td style={td}>
              <span style={{ background: u.role === 'admin' ? '#3498db' : '#eee',
color: u.role === 'admin' ? 'white' : 'black', padding: '0.2rem 0.6rem', borderRadius:
'12px', fontSize: '0.85rem' }}>
                {u.role}
              </span>
            </td>
            <td style={{ ...td, display: 'flex', gap: '0.5rem' }}>

```

```

                <button onClick={() => handleDeletarUsuario(u._id)} style={{ cursor:
'pointer', background: 'none', border: 'none', color: '#e74c3c' }}><Trash2
size={20}/></button>
            </td>
        </tr>
    </tbody>
</table>
</section>
</div>
)
}

```

```

const inputStyle = { width: '100%', padding: '0.6rem', borderRadius: '4px', border: '1px
solid #ccc' }
const th = { padding: '1rem', borderBottom: '2px solid #dee2e6' }
const td = { padding: '1rem', borderBottom: '1px solid #dee2e6' }

```

```
export default PainelAdm
```

A.21 frontend/src/App.css

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

```

```

body {
  background-color: #f4f4f9;
  color: #333;
}

```

```

.app-container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

```

```

.main-content {
  flex: 1;
  padding: 2rem;
  max-width: 1200px;
  margin: 0 auto;
  width: 100%;
}

```

A.22 frontend/src/index.css

```

:root {
  --text: #6b6375;
  --text-h: #08060d;
  --bg: #fff;
  --border: #e5e4e7;
  --code-bg: #f4f3ec;
  --accent: #aa3bff;
  --accent-bg: rgba(170, 59, 255, 0.1);
  --accent-border: rgba(170, 59, 255, 0.5);
  --social-bg: rgba(244, 243, 236, 0.5);
  --shadow:
    rgba(0, 0, 0, 0.1) 0 10px 15px -3px, rgba(0, 0, 0, 0.05) 0 4px 6px -2px;

  --sans: system-ui, 'Segoe UI', Roboto, sans-serif;
  --heading: system-ui, 'Segoe UI', Roboto, sans-serif;
  --mono: ui-monospace, Consolas, monospace;

  font: 18px/145% var(--sans);
  letter-spacing: 0.18px;
  color-scheme: light dark;
  color: var(--text);
}

```

```

background: var(--bg);
font-synthesis: none;
text-rendering: optimizeLegibility;
-webkit-font-smoothing: antialiased;
-moz-osx-font-smoothing: grayscale;

@media (max-width: 1024px) {
  font-size: 16px;
}

@media (prefers-color-scheme: dark) {
  :root {
    --text: #9ca3af;
    --text-h: #f3f4f6;
    --bg: #16171d;
    --border: #2e303a;
    --code-bg: #1f2028;
    --accent: #c084fc;
    --accent-bg: rgba(192, 132, 252, 0.15);
    --accent-border: rgba(192, 132, 252, 0.5);
    --social-bg: rgba(47, 48, 58, 0.5);
    --shadow:
      rgba(0, 0, 0, 0.4) 0 10px 15px -3px, rgba(0, 0, 0, 0.25) 0 4px 6px -2px;
  }

  #social .button-icon {
    filter: invert(1) brightness(2);
  }
}

body {
  margin: 0;
}

#root {
  width: 1126px;
  max-width: 100%;
  margin: 0 auto;
  text-align: center;
  border-inline: 1px solid var(--border);
  min-height: 100svh;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
}

h1,
h2 {
  font-family: var(--heading);
  font-weight: 500;
  color: var(--text-h);
}

h1 {
  font-size: 56px;
  letter-spacing: -1.68px;
  margin: 32px 0;
  @media (max-width: 1024px) {
    font-size: 36px;
    margin: 20px 0;
  }
}

h2 {
  font-size: 24px;
  line-height: 118%;
  letter-spacing: -0.24px;
  margin: 0 0 8px;
  @media (max-width: 1024px) {
    font-size: 20px;
  }
}

p {

```

```

    margin: 0;
  }

code,
.counter {
  font-family: var(--mono);
  display: inline-flex;
  border-radius: 4px;
  color: var(--text-h);
}

code {
  font-size: 15px;
  line-height: 135%;
  padding: 4px 8px;
  background: var(--code-bg);
}

```

A.23 frontend/vite.config.js

```

import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})

```

A.24 COMO _RODAR_EM_CASA.md

Como Continuar e Rodar o Projeto em Casa [CASA]

Este guia foi criado para ajudar você a rodar o projeto MERN (PAM) no seu próprio computador ou na sua casa, sem as restrições de rede que ocorrem no laboratório da FATEC.

Na FATEC, a rede bloqueia a conexão com o MongoDB Atlas por segurança (erro `querySrv ENODATA`). Em casa, na sua rede normal ou usando o 4G/5G do seu celular (roteador Wi-Fi do celular), esse bloqueio **não existe** e o projeto funcionará perfeitamente.

[CONFIG] Passo 1: Configuração Essencial no MongoDB Atlas (Muito Importante!)

Quando mudamos de rede (do laboratório para casa), o MongoDB Atlas bloqueia a conexão se o seu novo IP não estiver autorizado.

Para evitar isso, siga estes passos rápidos no painel do MongoDB Atlas:

1. Acesse o site do [MongoDB Atlas](https://cloud.mongodb.com/) e faça login.
2. No menu lateral esquerdo, em **Security**, clique em **Network Access**.
3. Clique no botão **Add IP Address** (Adicionar endereço IP) no canto superior direito.
4. Escolha uma das duas opções:
 - **Allow Access From Anywhere** (Permitir acesso de qualquer lugar - adiciona o IP `0.0.0.0/0`): **Recomendado para estudantes, pois funciona em qualquer rede que você estiver.**
 - Ou clique em **Add Current IP Address** (Adicionar IP atual) para liberar apenas a rede da sua casa.
5. Clique em **Confirm** e aguarde 1 minuto para o status ficar "Active".

[START] Passo 2: Como Iniciar o Projeto

Você precisará abrir dois terminais (linhas de comando) no seu VS Code, um para o Backend (servidor) e outro para o Frontend (React/Vite).

[BACKEND] Terminal 1: Iniciar o Servidor (Backend)

1. Certifique-se de estar na pasta do backend:


```

`bash
cd backend

```

```

    ```
2. Instale as dependências (caso mude de computador e não tenha a pasta `node_modules`):
    ```bash
    npm install
    ```
3. Inicie o servidor:
    ```bash
    npm start
    ```
4. Se der tudo certo, você verá a mensagem:
    ```
    [OK] Conectado ao MongoDB Atlas com sucesso!
    [START] Servidor rodando em http://localhost:5000
    ```

```

---

### [FRONTEND] Terminal 2: Iniciar o Site (Frontend)

```

1. Certifique-se de estar na pasta do frontend:
    ```bash
    cd frontend
    ```
2. Instale as dependências (se necessário):
    ```bash
    npm install
    ```
3. Inicie o React/Vite:
    ```bash
    npm run dev
    ```
4. Abra o navegador no endereço indicado (geralmente `http://localhost:5173`).

```

---

## [TESTE] Passo 3: Testando a Integração

Agora que tudo está rodando em casa:

```

1. Acesse `http://localhost:5173`
2. Vá em Criar Nova Conta (Cadastro) e preencha os dados de um novo usuário.
3. Se o cadastro for concluído com sucesso, significa que o frontend enviou os dados para o backend e ele salvou no MongoDB Atlas na nuvem!
4. Vá no painel do MongoDB Atlas, em Database -> Browse Collections, e você verá seu usuário salvo na coleção `usuarios`.
5. Vá no Painel do Administrador no frontend e adicione alguns produtos de teste para preencher a sua loja!

```

## A.25 implementation\_plan.md

# Plano: Estruturar e Conectar o Projeto MERN PAM ao MongoDB Atlas

## Contexto

O projeto é um e-commerce (PAM) com estrutura MERN:

```

- Frontend: React + Vite + React Router (porta 5173)
- Backend: Node.js + Express + Mongoose (porta 5000)

```

Atualmente faltam: conexão real com o Atlas, rotas de API, modelos Mongoose, script de start do backend, e as páginas do frontend ainda usam dados fictícios/alertas em vez de chamadas reais à API.

---

## Open Questions

```

> [!IMPORTANT]
> Você precisa fornecer sua MongoDB Atlas Connection String!
>
> A string tem este formato: `mongodb+srv://usuario:senha@cluster.mongodb.net/nome-do-banco`
>
> Você pode obtê-la no painel do Atlas em: Database → Connect → Drivers → Node.js
>
> Por segurança, ela será armazenada em um arquivo .env (nunca commitado no Git).

```

---

## ## Mudanças Propostas

### Backend (`c:\Users\240372\Desktop\PAM\backend`)

#### [NEW] `.env`

Arquivo com variáveis de ambiente (MONGO\_URI, PORT). Você preenche com sua string do Atlas.

#### [MODIFY] [server.js] (file:///c:/Users/240372/Desktop/PAM/backend/server.js)

- Reorganizar: carregar `dotenv` primeiro
- Usar `process.env.MONGO\_URI` para conexão
- Adicionar middleware `cors`
- Importar e usar as rotas de usuários e produtos

#### [NEW] `models/Usuario.js`

Schema Mongoose para usuários:

- `nome`, `usuario`, `email`, `senha` (hash), `endereco`, `telefone`, `cep`, `role` (admin/comum)

#### [NEW] `models/Produto.js`

Schema Mongoose para produtos:

- `nome`, `descricao`, `preco`, `imagem`, `estoque`

#### [NEW] `routes/usuarios.js`

Rotas REST:

- `POST /api/usuarios/cadastro` - Criar novo usuário
- `POST /api/usuarios/login` - Autenticar usuário
- `GET /api/usuarios` - Listar usuários (admin)
- `PUT /api/usuarios/:id` - Atualizar perfil
- `DELETE /api/usuarios/:id` - Deletar usuário (admin)

#### [NEW] `routes/produtos.js`

Rotas REST:

- `GET /api/produtos` - Listar todos os produtos
- `GET /api/produtos/:id` - Detalhe de um produto
- `POST /api/produtos` - Criar produto (admin)
- `PUT /api/produtos/:id` - Editar produto (admin)
- `DELETE /api/produtos/:id` - Deletar produto (admin)

#### [MODIFY] `package.json`

Adicionar script `"start": "node server.js"` e instalar `bcryptjs` para hash de senha.

---

### Frontend (`c:\Users\240372\Desktop\PAM\frontend`)

#### [NEW] `src/services/api.js`

Arquivo central para chamadas à API usando `fetch`, apontando para `http://localhost:5000`.

#### [MODIFY] [Login.jsx] (file:///c:/Users/240372/Desktop/PAM/frontend/src/pages/Login.jsx)

- Remover campo de "Confirmar Senha" (isso é cadastro, não login)
- Conectar ao endpoint `POST /api/usuarios/login`

#### [MODIFY]

[Cadastro.jsx] (file:///c:/Users/240372/Desktop/PAM/frontend/src/pages/Cadastro.jsx)

- Conectar ao endpoint `POST /api/usuarios/cadastro`

#### [MODIFY] [Home.jsx] (file:///c:/Users/240372/Desktop/PAM/frontend/src/pages/Home.jsx)

- Buscar produtos reais via `GET /api/produtos`

#### [MODIFY]

[ProdutoDetalhe.jsx] (file:///c:/Users/240372/Desktop/PAM/frontend/src/pages/ProdutoDetalhe.jsx)

- Buscar produto pelo ID via `GET /api/produtos/:id`

#### [MODIFY]

[PainelAdm.jsx] (file:///c:/Users/240372/Desktop/PAM/frontend/src/pages/PainelAdm.jsx)

- Buscar e gerenciar produtos/usuários reais via API

#### [MODIFY]

[Perfil.jsx] (file:///c:/Users/240372/Desktop/PAM/frontend/src/pages/Perfil.jsx)

- Conectar ao endpoint `PUT /api/usuarios/:id`

---

## Plano de Execução

1. Criar arquivo `.env` no backend (com placeholder para você preencher)
2. Instalar `bcryptjs` no backend
3. Criar modelos Mongoose (`Usuario.js`, `Produto.js`)
4. Criar rotas da API (`routes/usuarios.js`, `routes/produtos.js`)
5. Atualizar `server.js` com estrutura correta
6. Adicionar script `start` no `package.json` do backend
7. Criar `src/services/api.js` no frontend
8. Conectar as páginas do frontend às rotas reais
9. Iniciar backend (`npm start`) e frontend (`npm run dev`)

## Plano de Verificação

- Testar `GET http://localhost:5000/api/produtos` no navegador
- Cadastrar um usuário pelo frontend e ver no Atlas
- Verificar listagem de produtos no painel admin